

Université Paris Cité
Master 1 Computational Linguistics
Large Language Models — Lab 3

Where Hidden Rhythms in Silent Vectors Lie: A Character-Level GPT Model on Kazakh Poetry

Botakoz Kadyrbay

May 2026

GitHub repository: <https://github.com/bkadyrbay-hash/llm3lab>

1. Introduction and Corpus

This report describes the construction, training, and analysis of a character-level GPT model trained on a corpus of Kazakh poetry. The corpus consists of the collected works of Мағжан Жұмабаев (Magzhan Zhumabaev), one of the most important Kazakh poets of the early twentieth century, taken from the first volume of his collected works published in Almaty in 2013 by Жазушы. The volume contains poems and narrative poems (дастандар) spanning his entire creative career.

The corpus totals 571,006 characters across 25,263 lines, with a file size of approximately 994 KB. The text is written in Kazakh using the Cyrillic script and preserves line breaks, stanza boundaries, and all punctuation, which are essential for learning poetic structure. The vocabulary extracted from the corpus contains 142 unique characters, including Kazakh-specific letters (Ә, І, Ғ, Ғ, Ү, Ұ, Ө, Һ) alongside standard Cyrillic, punctuation, and a small number of Latin characters appearing in bibliographic metadata.

Kazakh classical poetry, and Zhumabaev in particular, is characterised by a consistent syllabic meter. The dominant form is the ымак (quatrain) with lines of 7 to 11 syllables, and a consistent rhyme scheme (AAAA or AABB). Choosing this corpus was motivated by the richness of its structural regularity, which makes it well suited for studying whether a small character-level model can internalise prosodic constraints.

2. Model Architecture

2.1 Overall design

The model is a decoder-only Transformer (GPT-style) operating at the character level. This design choice avoids the need for tokenisation and keeps the vocabulary small (142 characters), which substantially reduces computational cost. All architectural components were implemented in PyTorch using existing building blocks rather than low-level matrix operations. The baseline configuration is summarised in the following table.

Parameter	Value	Note
Context length (block_size)	256	Covers 3–4 typical stanzas of Kazakh verse (~160–200 chars each)
Number of layers	4	Baseline; varied in experiments
Hidden dimension (n_embd)	128	Compact but sufficient for a 142-char vocab
Attention heads	4	Head size = 32
MLP expansion	4×	FFN hidden = 512
Dropout	0.1	Applied in attention and FFN
Total parameters	861,070	~0.86M

2.2 Attention implementation

Two implementations of causal self-attention were written and compared. The first uses `torch.nn.functional.scaled_dot_product_attention` with `is_causal=True`, which automatically dispatches to FlashAttention on supported NVIDIA GPUs. The second is a classical implementation using a lower-triangular boolean mask materialised as a buffer:

```
wei = q @ k.transpose(-2, -1) * head_size**-0.5
wei = wei.masked_fill(tril[:,T, :T] == 0, float('-inf'))
wei = F.softmax(wei, dim=-1)
```

Both implementations were benchmarked on 50 forward passes with the baseline configuration (batch=32, ctx=256) on a T4 GPU. The results are shown below.

Implementation	Time per forward (ms)	Peak GPU memory (MB)
FlashAttention (SDPA)	25.74	64.6
Manual masked attention	16.00	68.6

Somewhat counterintuitively, the manual implementation is faster at this sequence length. This is expected: FlashAttention's main advantage is memory efficiency through tiling, which avoids materialising the full $N \times N$ attention matrix. At context length 256, the attention matrix is only 256×256 , small enough to fit in L2 cache for both implementations. The memory saving of FlashAttention (+4 MB less) confirms the theoretical expectation. The performance benefit of FlashAttention becomes significant at context lengths above ~1,000 tokens, which is beyond the scope of this project. For the actual training runs, the SDPA implementation was used throughout for consistency.

2.3 Positional embeddings

Learned positional embeddings were chosen over sinusoidal encodings and RoPE. The rationale is straightforward: sinusoidal embeddings have the theoretical advantage of generalising to sequence lengths not seen during training, but since context length is fixed at 256 throughout this project, this advantage does not apply. RoPE modifies attention score computation and adds implementation complexity without a clear benefit at this scale. Learned embeddings are the simplest option and have been shown to perform comparably to sinusoidal encodings in GPT-style models at similar scales. The embedding table is an `nn.Embedding` of size (256, 128).

2.4 Feed-forward activation

The feed-forward sublayer uses GELU rather than ReLU. GELU is a smooth approximation to ReLU and has become the standard choice in language model feed-forward layers since it was adopted in GPT-2. In practice the difference in loss is small, but the choice is more consistent with current practice in the field.

3. Training

3.1 Setup

The model was trained for 20,000 steps using the AdamW optimiser with a learning rate of 3×10^{-4} and a batch size of 32. The data was split 90/10 into training and validation sets. Training was conducted on a Google Colab T4 GPU and completed in approximately 90 minutes.

Training and validation loss were evaluated every 1,000 steps using a dedicated `estimate_loss()` function that averages cross-entropy over 200 random batches with dropout disabled (`model.eval()`). Bits-per-character (BPC), defined as `val_loss / log(2)`, was recorded alongside the losses. Short text samples were generated from the prompt "Желсіз түнде жарык ай\n" every 2,000 steps to monitor qualitative progress.

3.2 Learning curves

The final training statistics are shown below. The full loss curves are available in the accompanying notebook.

Step	Train loss	Val loss	BPC
0	5.0279	5.0384	7.27
1,000	2.2995	2.3453	3.38
5,000	1.7824	1.9205	2.77
10,000	1.5512	1.7626	2.54
15,000	1.4335	1.7234	2.49
19,000	1.3661	1.7128	2.47

Training loss drops steeply in the first 3,000 steps, indicating rapid learning of character-level statistics. The gap between train and validation loss stabilises around step 10,000, with both curves flattening. Validation loss at step 15,000 (1.7234) is almost identical to step 19,000 (1.7128), suggesting that 20,000 steps is near the practical optimum for this model size

and corpus. There is no strong evidence of overfitting: the train–val gap at step 19,000 is 0.35, which is moderate and stable.

The final BPC of 2.47 is the primary compression metric. For reference, a model that assigns equal probability to all 142 characters would achieve $\text{BPC} = \log_2(142) \approx 7.15$. The trained model compresses to 2.47 BPC, indicating that it has learnt substantial statistical regularity in the text.

3.3 Qualitative evolution during training

The generated samples provide a complementary view of learning progress. At step 2,000, output consists of partially-formed Kazakh words with mostly correct morphological endings but no coherent syntax. By step 8,000, the model produces recognisable Kazakh words and short phrases, and line breaks appear at plausible positions. By step 12,000, multi-word phrases are grammatically plausible and occasional 4-line stanza-like structures emerge. This progression from character statistics to word-level and then structural patterns is consistent with what the assignment described as "hidden rhythms begin to form."

3.4 Implementation difficulties and how they were resolved

Several issues arose during the corpus preparation, implementation, and training process.

The first difficulty was corpus size. The initial corpus consisted of a small collection of Kazakh poems totalling approximately 550 KB — below the 1–2 MB minimum recommended for exploratory experiments. The model trained on this version produced very repetitive output and showed signs of memorisation rather than generalisation, as the training loss dropped much faster than the validation loss. To address this, the corpus was expanded by adding the full first volume of Мағжан Жұмабаев's collected works, bringing the total to 571,006 characters (~994 KB). The expanded corpus produced noticeably more varied generated text and a more stable train–val gap.

The second issue was a missing `estimate_loss()` function: the initial version of the training loop called this function before it was defined, causing a `NameError` on the first evaluation step. The fix was to define the function before the training loop, and to ensure it calls `model.eval()` before computing losses (to disable dropout) and `model.train()` afterwards. Without this, dropout remains active during evaluation, which artificially lowers the measured validation loss and makes overfitting impossible to detect reliably.

The second issue concerned global state in hyperparameter experiments. The original notebook reassigned `n_layer` and `block_size` as global variables before creating a new model, which meant that any cell re-run out of order could silently use the wrong configuration. The solution was to pass all parameters explicitly to the `GPTPoetry` constructor and to define a dedicated `run_experiment()` function with its own local `get_batch()` closure that respects the experiment-specific `block_size`.

A third difficulty was the Flash vs Manual attention benchmark producing a counterintuitive result: the manual implementation (16.0 ms) was faster than FlashAttention (25.7 ms) on the T4 GPU. After reviewing the PyTorch documentation, this was understood as expected behaviour: at context length 256 the $N \times N$ attention matrix is small enough to remain in the GPU L2 cache, eliminating the memory bandwidth bottleneck that FlashAttention is designed to address. The benchmark result is therefore not an error but a meaningful finding about the regime in which each implementation excels.

4. Hyperparameter Experiments

Two hyperparameters were varied against the baseline, each trained to 20,000 steps under otherwise identical conditions to ensure a fair comparison.

Experiment 1 — Number of layers: 4 → 2

Halving the number of Transformer blocks reduces model capacity. Each block applies one round of multi-head attention followed by a feed-forward sublayer; with two blocks, the model has fewer compositional stages to build up representations from individual characters to syllables and words. The final validation loss of 1.8574 (BPC 2.68) is 0.14 higher than the baseline, confirming that depth matters even at this small scale.

Interestingly, the 2-layer model converges more smoothly — its val loss curve decreases monotonically without any upward fluctuations — because its smaller capacity makes it less prone to locally overfitting individual batches. This illustrates the classic capacity–stability trade-off in neural networks.

Experiment 2 — Context length: 256 → 128

A shorter context window limits the model to 128 characters of history. In Жұмабаев's verse, a typical 4-line stanza spans approximately 160–200 characters. With $\text{ctx}=128$, the model cannot see a complete quatrain within a single context window, which directly impairs its ability to learn stanza-level patterns and cross-line rhyme.

The final validation loss of 1.7470 (BPC 2.52) is only marginally worse than the baseline (1.7128, BPC 2.47). This suggests that on a character-level task the most predictable regularities — common suffixes, word boundaries, letter co-occurrences — are already captured within 128 characters. The stanza-level structure, which requires longer context, is harder to measure via cross-entropy alone; its absence becomes more apparent in the structural analysis of generated text (Section 5).

Configuration	Final val loss	BPC	Δ vs baseline
Baseline: 4L, $\text{ctx}=256$	1.7128	2.471	—
Exp 1: 2L, $\text{ctx}=256$	1.8574	2.680	+0.144
Exp 2: 4L, $\text{ctx}=128$	1.7470	2.520	+0.034

The comparison shows that depth has a larger effect on loss than context length, at least within the ranges tested. Both experiments confirm that the baseline configuration represents a reasonable compromise between model size, context coverage, and computational cost.

5. Analysis of Generated Poetry

5.1 Sample poems

The following excerpt was generated at temperature 0.6 from the prompt "Желсіз түнде жарық ай\n" (the opening line of famous poem):

Желсіз түнде жарық ай

*Ұлы да жамылған жоқ ойнап.
Жас келін тапса есін,*

*Жарықты көрген жас келеді.
Айдар жарық бала жаным,
Есікте айға қара жаным,
Жас құладым да жерден шықпай
Кірген жаным да күледі.*

*Есіме алмай жүректің,
Сондықтан жаным да алмай жүрмей ме?!*

The text is syntactically well-formed Kazakh: verb endings, case suffixes, and postpositions are used correctly. The word "жаным" (my soul/dear) appears repeatedly, which is authentic to Zhumabaev style — it is one of the most frequent words in his corpus. Line breaks occur at plausible syntactic boundaries. However, semantic coherence across lines remains limited: the poem reads as a series of disconnected images rather than a unified narrative.

At temperature 0.9, the output is more varied in vocabulary but loses stanza structure, with irregular line lengths. At top-k=5 (temperature 0.8), the model becomes repetitive, cycling through a small set of high-probability continuations.

5.2 Structural properties

The following metrics compare the training corpus to the generated text (temperature 0.9, 500 new tokens):

Metric	Corpus	Generated	Assessment
Avg line length (chars)	27.0	33.9	Generated lines ~25% longer
Avg stanza length (lines)	13.96*	15.00*	Unreliable metric — see note below
Avg syllables per line	9.07	11.20	Generated slightly above corpus range
Top-20 ending overlap	—	1/20	Rhyme endings not well reproduced

Stanza length — methodological note

The `stanza_lengths()` function splits text on double newlines (`\n\n`). In the corpus, individual poems are separated by `\n\n`, while stanza boundaries within a poem use single `\n`. As a result, the function treats each entire poem as a single "stanza," producing a meaningless average of ~14 lines per "stanza" for the corpus and a single block of 15 lines for the short generated sample. The metric is therefore not interpretable and is flagged with an asterisk in the table. A corrected approach would require poem-level segmentation of the corpus before stanza analysis.

Line length

Generated lines average 33.9 characters versus the corpus mean of 27.0. The discrepancy suggests the model has not fully internalised the line-break positions, likely because at 500 generated tokens the sample is too short for stable statistics. At lower temperatures, where the model is more conservative, line lengths align more closely with the corpus.

Stanza length — methodological note

The `stanza_lengths()` function splits text on double newlines (`\n\n`). In the corpus, individual poems are separated by `\n\n`, while stanza boundaries within a poem use single `\n`. As a result, the function treats each entire poem as one "stanza," giving a meaningless average of ~14 lines per "stanza" for the corpus and a single block for the short generated sample. The metric is flagged with an asterisk in the table. A corrected approach would require poem-level segmentation of the corpus before stanza analysis — something to address in future work.

Rhyme endings

The rhyme analysis reveals an important limitation. Only 1 of the top-20 most common line endings in the generated text matches the top-20 in the corpus. In the corpus, the dominant endings (аған, айды, анда, екен) reflect the characteristic agglutinative suffixes of Kazakh verbal and nominal forms. In the generated text, endings are much more varied, with no single ending appearing more than twice in the 500-token sample. This indicates that the model has not learnt to maintain consistent rhyme schemes across stanzas, which requires planning several lines ahead — a capability that goes beyond what character-level next-token prediction can easily achieve at this scale.

This finding is consistent with the context length experiment: even with `ctx=256` (covering roughly one full stanza), generating rhyming quatrains requires the model to anticipate the rhyme at the end of line 4 when generating line 1, which is a form of long-range dependency that cross-entropy training on next characters does not directly incentivise.

5.3 Summary

The model successfully learnt low-level Kazakh linguistic structure: valid characters, correct morphological endings, plausible word boundaries, and approximate syllabic meter. It did not consistently learn high-level poetic structure: cross-line rhyme patterns and stanza organisation are largely absent. This is precisely the boundary between what character-level statistics can capture and what requires explicit structural supervision or a much larger model. The findings are interpretable and consistent across the quantitative metrics and the qualitative reading of the generated text.

6. Conclusion

This project built and trained a character-level GPT model with 861K parameters on a 571K-character corpus of Kazakh classical poetry by Мағжан Жұмабаев. The model reached a validation loss of 1.71 (BPC 2.47) after 20,000 training steps. Hyperparameter experiments confirmed that model depth has a larger impact on loss than context length within the tested ranges, and that the baseline configuration (4 layers, `ctx=256`) represents a sensible default.

The analysis of generated text shows that linguistic structure emerges at two different levels. At the character and morpheme level, the model reliably produces valid Kazakh: correct suffixes, plausible words, and syntactically well-formed short phrases. At the stanza level, rhyme and consistent line-length patterns are largely absent. This two-tier picture is not a failure of the model but an illustration of which structures are recoverable from local character statistics and which require longer-range dependencies or explicit structural objectives.

The choice of Жұмабаев's corpus proved well motivated: the regularity of his syllabic metre is detectable in the generated text even without explicit supervision, whereas the more demanding cross-line rhyme patterns are not. The gap between these two levels is precisely where "hidden rhythms begin to form" — and where they still have further to go.

7. Code and Reproducibility

The full implementation is available in the Jupyter notebook `lab3_magzhan_final.ipynb`. To reproduce the results, upload the corpus file (`poems.txt`, Мағжан Жұмабаев collected poems), run all cells in order, and ensure a GPU runtime is selected in Google Colab. All random seeds are not fixed, so minor numerical differences in loss values are expected across runs.

— *End of report* —